



微控制器（STM32/LiteOS）课程 逐页精讲与复习全解

第一部分 课程说明

1. 课程信息

- **课程名称：**微控制器（STM32系列，ARM Cortex-M3内核）
- **授课教师：**毛老师（重庆大学电气工程学院） myx@cqu.edu.cn
- **MOOC平台：**<https://cquv3.xuetangx.com>
- **教材：**自印讲义 + 《ARM Cortex-M3嵌入式微控制器原理与应用(STM32系列)》清华大学出版社

2. 成绩构成

项目	占比	说明
实验	30分	上机考15分 + 平时实验10分 + 实验报告5分
理论	70分	期末考试50分 + 平时作业10分 + 课堂表现5分 + 在线测试5分
合计	100分	

3. 课程内容分布

章节	内容	学时
1	微机原理基础知识	8
2	ARM体系结构与CM3内核	12

章节	内容	学时
3	STM32最小系统与开发环境	4
4	STM32资源与功能部件	16
5	LiteOS系统基础	2
6	LiteOS任务概述	4
7	LiteOS任务实现	2
8	实验	32

4. 课程要求

- 旷课达3次取消考试资格
- 考试必须携带学生证
- 每次实验前需预习并准备代码
- 每次实验需提交实验报告

第二部分 微型计算机基础

1. 微型计算机硬件系统组成

1.1 三大核心部件

部件	功能	组成
CPU（中央处理器）	运算核心、控制系统"大脑"	ALU + CU + 寄存器组
存储器	存储程序和数据的空间	ROM + RAM
I/O接口	与外部设备通信	并行/串行接口

1.2 CPU内部结构

- **ALU（算术逻辑单元）**：执行算术和逻辑运算
 - 累加器Accumulator(A)：存放操作数和运算结果
 - 暂存器TMP：暂时存放操作数
 - 标志寄存器F：存放运算结果的状态标志（进位、零、负数等）
- **CU（控制单元）**：指令译码、产生控制信号
- **寄存器组（Registers）**：高速存储单元

1.3 总线

- **数据总线（DB）**：传输数据，双向
- **地址总线（AB）**：传输地址，单向（CPU→存储器/I/O）
- **控制总线（CB）**：传输控制信号

例题1： CPU中包含哪些主要部件？各有什么功能？

答案： ①ALU：执行算术逻辑运算；②CU：指令译码与控制；③寄存器组：高速暂存数据。
具体包括累加器(A)、暂存器(TMP)、ALU、标志寄存器(F)等。

例题2： 微型计算机的三总线结构是什么？

答案： 数据总线DB（双向传数据）、地址总线AB（单向传地址）、控制总线CB（传控制信号）。

2. 数制与编码

2.1 常用数制

数制	基数	数码	后缀
二进制	2	0,1	B
八进制	8	0~7	O/Q

数制	基数	数码	后缀
十进制	10	0~9	D
十六进制	16	0 ₉ ,A _F	H

2.2 进制转换

- **二→十六**：每4位二进制对应1位十六进制
- **十六→二**：每位十六进制对应4位二进制
- **十→二**：除2取余法（整数部分），乘2取整法（小数部分）

2.3 有符号数的表示

- **原码**：最高位为符号位（0正1负），其余为数值位
- **反码**：正数同原码，负数符号位不变其余取反
- **补码**：正数同原码，负数反码+1
 - **计算机中统一使用补码表示有符号数**

例题1：将十进制数 -25 转换为8位二进制补码表示。

解：

① 25的二进制原码：00011001B

② 取反：11100110B

③ +1：11100111B

答案：11100111B（补码）

例题2：十六进制数 3A7FH 转换为二进制数。

解：3→0011，A→1010，7→0111，F→1111

答案：0011 1010 0111 1111B

3. 基本逻辑运算

运算	符号	规则
与(AND)	$\& / \cdot$	全1则1, 有0则0
或(OR)	$ / +$	全0则0, 有1则1
非(NOT)	$\sim / \neg$	取反
异或(XOR)	$\wedge / \oplus$	相同为0, 不同为1

按位操作应用

- **AND**: 可用于清0指定位 (与0相与)
- **OR**: 可用于置1指定位 (与1相或)
- **XOR**: 可用于翻转指定位 (与1异或)
- **NOT**: 按位取反

例题1: 给定 $A = 0011B$, $B = 0101B$, 求 $A \& B$ 、 $A | B$ 、 $A \wedge B$ 、 $\sim A$ 。

解:

- $A \& B = 0011 \& 0101 = 0001B$
- $A | B = 0011 | 0101 = 0111B$
- $A \wedge B = 0011 \wedge 0101 = 0110B$
- $\sim A = 1100B$

例题2: 如何用位运算将寄存器某位置1? 如何清0?

解:

- **置1**: 用 OR 操作, $REG |= (1 \ll n)$
 - **清0**: 用 AND 操作, $REG \&= \sim(1 \ll n)$
 - **翻转**: 用 XOR 操作, $REG \wedge= (1 \ll n)$
-

4. 存储器组织

4.1 存储容量

- 1 Byte = 8 bit
- 1 KB = 2^{10} = 1024 Byte
- 1 MB = 2^{20} Byte

4.2 存储地址范围计算

- n根地址线 → 寻址范围：0 ~ $2^n - 1$
- 地址范围 = 容量（以字节为单位）

例题：某计算机有20根地址线，其最大寻址范围是多少？

解： 2^{20} = 1MB

答案：1MB

第三部分 Cortex-M3 基础

1. ARM Cortex-M3 处理器特点

- 32位RISC处理器
- 哈佛架构：指令总线与数据总线分离
- Thumb-2指令集：16/32位混合指令
- 三级流水线：取指→译码→执行
- NVIC（嵌套向量中断控制器）：低延迟中断响应
- 低功耗设计

2. CM3 寄存器组

2.1 通用寄存器（R0~R15）

寄存器	别名	功能
R0~R7	—	低组通用寄存器，所有指令均可访问
R8~R12	—	高组通用寄存器，部分指令可访问
R13	MSP/PSP	堆栈指针（主堆栈/进程堆栈）
R14	LR	链接寄存器，保存返回地址
R15	PC	程序计数器，指向当前指令+4

2.2 特殊功能寄存器

寄存器	功能
xPSR	程序状态寄存器（含APSR、IPSR、EPSR）
PRIMASK	关闭除NMI和HardFault外所有中断
FAULTMASK	关闭除NMI外所有中断
BASEPRI	屏蔽优先级低于某值的中断
CONTROL	控制堆栈选择（MSP/PSP）及特权级

例题：Cortex-M3中哪些寄存器可以访问存储器？哪个寄存器是堆栈指针？

答案：R0~R12可进行存储器访问。R13是堆栈指针，分为MSP（主堆栈指针）和PSP（进程堆栈指针）。

3. 堆栈与堆栈操作

3.1 堆栈类型

- **满递减堆栈**：SP指向最后一个压入的数据，压栈时先减SP再存数据
- CM3默认使用**满递减堆栈**

3.2 压栈与出栈

- **PUSH {reg}**: $SP \leftarrow SP - 4$, $[SP] \leftarrow reg$
- **POP {reg}**: $reg \leftarrow [SP]$, $SP \leftarrow SP + 4$

3.3 异常进入时的自动压栈

当异常发生时，硬件自动将 **xPSR, PC, LR, R12, R3~R0** 压栈

例题：Cortex-M3在异常响应时，硬件自动将哪些寄存器压栈？

答案：xPSR、PC、LR、R12、R3、R2、R1、R0，共8个寄存器，占用32字节堆栈。

4. 中断与异常

4.1 异常类型

异常号	优先级	名称
1	-3	Reset
2	-2	NMI
3	-1	HardFault
4~10	可配置	MemManage/BusFault/UsageFault
11	可配置	SVCall
12	可配置	Debug Monitor

异常号	优先级	名称
14	可配置	PendSV
15	可配置	SysTick
16+	可配置	外部中断IRQ

4.2 NVIC 中断优先级

- **优先级寄存器**：每个中断源对应8位优先级寄存器，但具体使用多少位由芯片厂商决定
- **STM32F103**：使用 **4位** 优先级（高4位有效），可配置16级优先级
- **AIRCR寄存器**（SCB->AIRCR）的8:10位用于分组
 - **抢占优先级** 和 **子优先级** 的分组

4.3 中断优先级分组

AIRCR[10:8]	分组	抢占优先级位数	子优先级位数
000	0	0	4
001	1	1	3
010	2	2	2
011	3	3	1
100	4	4	0

4.4 中断优先级比较

- 数值**越小**，优先级**越高**
- 先比较**抢占优先级**，抢占优先级高的可以打断抢占优先级低的中断
- 抢占优先级相同，再比较**子优先级**，但不能互相打断（需等待当前中断完成）
- 若完全相同，则比较**中断号**（中断号小的优先级高）

例题1：NVIC->IPR[1]对应中断4、5、6、7的优先级设置，若该寄存器值为 0x70806050，则这四个中断的优先级顺序是什么？

解：0x70806050 → 按字节解析（IPR[n]的[7:0]对应最低号中断）：

- [7:0] = 0x50 → 中断4, 优先级80
- [15:8] = 0x60 → 中断5, 优先级96
- [23:16] = 0x80 → 中断6, 优先级128
- [31:24] = 0x70 → 中断7, 优先级112

数值越小优先级越高: 80(IRQ4) < 96(IRQ5) < 112(IRQ7) < 128(IRQ6)

答案: 中断4 > 中断5 > 中断7 > 中断6

例题2: AIRCR寄存器中, 优先级分组设为分组3 (AIRCR[10:8]=011, 即抢占优先级3位、子优先级1位), 求某中断的抢占优先级为3、子优先级为1时, 优先级寄存器的值。

解: 当抢占优先级3位, 子优先级1位时

- 抢占优先级3, 子优先级1 → 二进制: 011 1 → 0x70

答案: 0x70

5. 存储器映射与大小端

5.1 CM3存储器映射

地址范围	区域	用途
0x00000000~0x1FFFFFFF	Code	代码区 (Flash)
0x20000000~0x3FFFFFFF	SRAM	静态RAM区
0x40000000~0x5FFFFFFF	Peripheral	片上外设区
0x60000000~0x9FFFFFFF	External RAM	外部RAM
0xA0000000~0xDFFFFFFF	External Device	外部设备
0xE0000000~0xFFFFFFFF	System	系统级控制 (NVIC等)

5.2 大小端模式

模式	特点	示例 (0x12345678存于0x20000000)
大端	高位字节存低地址	0x20000000: 12, 0x20000001:34
小端	低位字节存低地址	0x20000000: 78, 0x20000001:56
STM32默认	小端模式	—

例题：采用小端格式在地址2000H处存放32位数据0x12345678，则2000H、2001H、2002H、2003H处的字节各是多少？

解：小端：低地址存低位字节

- 2000H: 78H
- 2001H: 56H
- 2002H: 34H
- 2003H: 12H

答案：78H, 56H, 34H, 12H

6. 位带操作 (Bit-Banding)

- 将位操作映射到字操作
- 位带区：SRAM (0x20000000~0x200FFFFFF) 和 外设 (0x40000000~0x400FFFFFF)
- 位带别名区：每个位映射到一个32位字地址

位带映射公式

$$\text{别名区地址} = \text{位带区基址} + \text{字节偏移} \times 32 + \text{位号} \times 4$$

例题：SRAM位带区地址0x20000000的第0位映射到位带别名区的哪个地址？

解：SRAM位带基址对应别名基址0x22000000

$$\text{别名地址} = 0x22000000 + 0 \times 32 + 0 \times 4 = 0x22000000$$

答案：0x22000000

第四部分 STM32最小系统与开发环境

1. STM32最小系统组成

- **电源电路**：VDD=3.3V，VSS=GND
- **晶振电路**：HSE（外部高速时钟）+ LSE（外部低速时钟32.768kHz）
- **复位电路**：NRST引脚，低电平复位
- **调试接口**：SWD（SWDIO/SWCLK）或 JTAG
- **启动模式选择**：BOOT0, BOOT1

启动模式

BOOT0	BOOT1	启动区域
0	X	主Flash
1	0	系统存储器（Bootloader）
1	1	内嵌SRAM

例题：STM32最小系统至少需要哪些电路？若BOOT0=0，BOOT1=1，从何处启动？

答案：①电源电路、②晶振电路（HSE+LSE）、③复位电路、④调试接口（SWD/JTAG）、⑤启动模式选择（BOOT引脚）。

BOOT0=0 → **从主Flash启动。**

2. STM32时钟系统

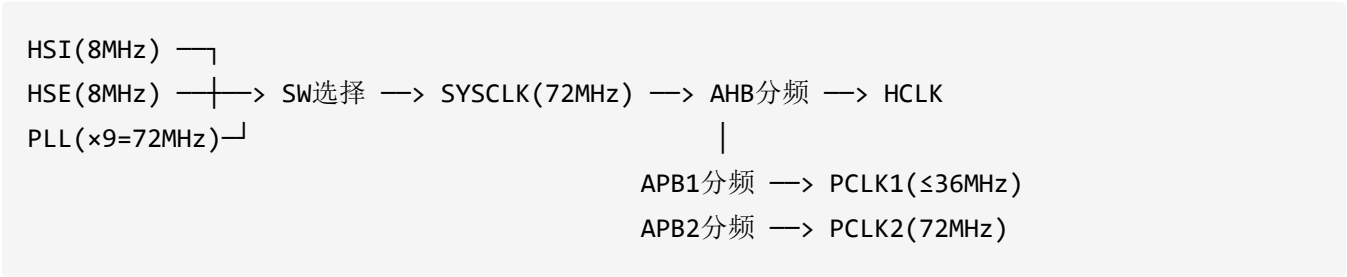
2.1 时钟源

时钟源	频率	说明
HSI	8MHz	内部高速RC振荡器
HSE	4~16MHz	外部高速晶振
LSI	40kHz	内部低速RC
LSE	32.768kHz	外部低速晶振（RTC用）
PLL	—	锁相环，倍频输出

2.2 系统时钟（SYSCLK）来源

RCC_CFGR[1:0] (SW)	SYSCLK来源
00	HSI
01	HSE
10	PLL输出

2.3 典型时钟树



2.4 AHB预分频 (HPRE)

RCC_CFGR[7:4]	AHB分频系数
0xxx	SYSCLK不分频
1000	SYSCLK / 2
1001	SYSCLK / 4
1010	SYSCLK / 8
1011	SYSCLK / 16
1100	SYSCLK / 64
1101	SYSCLK / 128
1110	SYSCLK / 256
1111	SYSCLK / 512

2.5 APB预分频

- **PPRE1 (APB1)** : RCC_CFGR[10:8]
 - 0xx: HCLK不分频
 - 100: HCLK/2, 101: /4, 110: /8, 111: /16
 - APB1最高频率 **36MHz**
- **PPRE2 (APB2)** : RCC_CFGR[13:11]
 - APB2最高频率 **72MHz**

2.6 CSS (时钟安全系统)

- 监测 **HSE** 是否失效
- 若HSE失效, 自动切换至 **HSI**
- 同时产生 **NMI** 中断

例题1: 已知SYSCLK频率为72MHz, 若AHB预分频系数为3 (即HPRE[7:4]=0011), 则HCLK频率为?

解: HPRE=0xxx时AHB不分频, 所以HCLK=72MHz

答案：72MHz

例题2：已知SYSCLK=72MHz，若HPRE[7:4]=1011（SYSCLK/16），APB1不分频，求HCLK和PCLK1频率。

解：

$$HCLK = 72\text{MHz} / 16 = 4.5\text{MHz}$$

$$PCLK1 = HCLK \text{（不分频）} = 4.5\text{MHz}$$

答案：HCLK=4.5MHz, PCLK1=4.5MHz

例题3：CSS（时钟安全系统）的作用是什么？当检测到HSE失效时会如何？

答案：CSS用于监测HSE（外部高速晶振），当HSE失效时自动切换系统时钟到HSI（内部8MHz），并产生NMI中断通知CPU。

第五部分 开发环境使用与GPIO操作

1. GPIO 工作模式

模式	输出类型	说明
浮空输入	—	无上拉/下拉，引脚高阻
上拉输入	—	内部上拉电阻接到VDD
下拉输入	—	内部下拉电阻接到VSS
模拟输入	—	用于ADC，关闭数字功能
推挽输出	Push-Pull	可输出高低电平，驱动能力强
开漏输出	Open-Drain	只能输出低电平高阻，需外接上拉
复用推挽	AF Push-Pull	由片上外设控制输出
复用开漏	AF Open-Drain	由片上外设控制（如I2C）

2. GPIO 配置寄存器

寄存器	功能	位数
GPIOx_CRL	低8位引脚模式 (Pin0~7)	32位, 每配置4位
GPIOx_CRH	高8位引脚模式 (Pin8~15)	32位, 每配置4位
GPIOx_IDR	输入数据寄存器	仅读
GPIOx_ODR	输出数据寄存器	可读写
GPIOx_BSRR	位设置/复位寄存器	写1置位/复位
GPIOx_BRR	位复位寄存器	写1复位
GPIOx_LCKR	锁定寄存器	锁定配置

每4位配置格式 (CRL/CRH)

[1:0] 模式位	功能	[3:2] CNF位	输入模式	输出模式
00	输入模式	00	模拟输入	—
01	输出(10MHz)	01	浮空输入	推挽输出
10	输出(2MHz)	10	上拉/下拉	开漏输出
11	输出(50MHz)	11	保留	复用功能开漏

3. GPIO 输出操作

```
// 设置GPIOB Pin5 为推挽输出, 50MHz
GPIOB->CRL &= ~(0x0F << 20); // 清零第5引脚配置位(bit[23:20])
GPIOB->CRL |= (0x03 << 20); // 设置MODE=11(50MHz), CNF保持为00(推挽)
// 注: 每引脚4位 = CNF[1:0](bit[23:22]) + MODE[1:0](bit[21:20]), 清零后CNF默认为00

// 方式1: 操作ODR
GPIOB->ODR |= (1 << 5); // 置高
GPIOB->ODR &= ~(1 << 5); // 置低

// 方式2: 操作BSRR (推荐, 原子操作)
GPIOB->BSRR = (1 << 5); // 置高 (BSRRL)
GPIOB->BSRR = (1 << (5+16)); // 置低 (BSRRH)
```

4. GPIO 输入操作

```
// 读取输入
uint8_t val = (GPIOA->IDR >> 0) & 1; // 读取PA0引脚电平
```

例题1: 要将PA0配置为浮空输入模式, 写出配置代码。

解:

```
RCC->APB2ENR |= (1 << 2); // 开启GPIOA时钟
GPIOA->CRL &= ~0x0F; // 清零PA0的CNF0[1:0]和MODE0[1:0]
// MODE[1:0]=00(输入), CNF[1:0]=01(浮空), 即0001→0x1
GPIOA->CRL |= 0x04; // 设置为浮空输入 (0x4=0100: MODE=00, CNF=01)
```

例题2: 使用BSRR寄存器将PB0置1, PB1置0, 写出代码。

解:

```
// PB0置1 (BSRRL的bit0写1)
GPIOB->BSRR = (1 << 0);
// PB1置0 (BSRRH的bit(16+1)写1)
GPIOB->BSRR = (1 << (16 + 1));
// 或一次操作:
GPIOB->BSRR = (1 << 0) | (1 << (16 + 1));
```

例题3：简述推挽输出和开漏输出的区别。

答案：推挽输出可主动输出高低电平，由P-MOS和N-MOS交替导通；开漏输出只能主动拉低（N-MOS导通），输出高电平时为高阻态，需外接上拉电阻。开漏输出可实现"线与"功能（多个引脚直接并联）。

- 推挽：驱动能力强，适合驱动LED、数字信号输出
- 开漏：适合多路共用总线（如I2C），电平转换

第六部分 STM32功能部件及应用

1. 定时器（TIMER）

1.1 定时器分类

类型	定时器	位数	特点
基本定时器	TIM6,TIM7	16位	仅基本定时功能
通用定时器	TIM2,TIM3,TIM4,TIM5	16/32位	含PWM、输入捕获、输出比较
高级定时器	TIM1,TIM8	16位	含死区、刹车、互补输出

1.2 定时器组成

- **预分频器(PSC):** 对时钟源分频，16位
- **计数器(CNT):** 向上/向下/中心对齐计数模式

- **自动重载寄存器(ARR)**: 计数周期设定
- **计数频率公式**:

计数器频率 = 定时器时钟 / (PSC + 1)

溢出时间 = (ARR + 1) × (PSC + 1) / 定时器时钟

△ **关键注意**: STM32F103中, 定时器时钟 (TIMxCLK) 取决于APB预分频系数:

- 若APBx预分频 = 1 → TIMxCLK = PCLKx
- 若APBx预分频 ≠ 1 → TIMxCLK = **PCLKx × 2**

例如SYSCLK=72MHz时, APB1需/2分频得PCLK1=36MHz (≠1), 此时TIM2~7时钟 = 36MHz × 2 = 72MHz。

以下例题假设APBx预分频=1或已直接给定定时器时钟频率。

例题1: APB1时钟为36MHz, TIM2预分频系数PSC=35999, ARR=999, 求TIM2的溢出时间。

解:

计数器频率 = 36MHz / (35999 + 1) = 36MHz / 36000 = 1000Hz = 1kHz

溢出时间 = (999 + 1) / 1000Hz = 1s

答案: 1秒

例题2: 若要使TIM3每1ms产生一次中断, PCLK1=36MHz, 求PSC和ARR的合理取值 (PSC=某值, ARR=某值)。

解:

溢出时间 = (ARR+1)×(PSC+1) / 36MHz = 0.001s

取 PSC = 35999 → 计数频率 = 36M/36000 = 1kHz

ARR = 0.001 × 1kHz - 1 = 0

答案: PSC=35999, ARR=0 (计数1次, 1ms)

或 PSC=7199 → 计数频率 = 36M/7200 = 5kHz

ARR = 0.001 × 5000 - 1 = 4

答案: PSC=7199, ARR=4

2. PWM 输出

2.1 PWM参数

参数	定义
频率	$1 / \text{周期}$
占空比	$\text{高电平时间} / \text{周期} \times 100\%$
分辨率	$1 / \text{ARR} \times 100\%$

2.2 PWM模式

- **模式1**: $\text{CNT} < \text{CCR}_x$ 时输出有效电平
- **模式2**: $\text{CNT} > \text{CCR}_x$ 时输出有效电平

2.3 PWM配置步骤

1. 开启定时器时钟和GPIO时钟
2. 配置GPIO为复用推挽输出 (AF_PP)
3. 配置定时器: PSC、ARR、CCR_x、CCMR_x (模式)
4. 配置CCER (极性、使能)
5. 配置BDTR (高级定时器需要)
6. 使能自动重装载预装载
7. 使能定时器

例题1: PCLK1=36MHz, 使用TIM3产生频率为1kHz、占空比为50%的PWM, 求PSC和CCR3。

解:

频率 = 1kHz → 周期 = 1ms
取 PSC = 3599 → 计数频率 = 36M/3600 = 10kHz
ARR = 1ms × 10kHz - 1 = 9
占空比50% → CCR3 = ARR × 50% = 5

答案: PSC=3599, ARR=9, CCR3=5

例题2：STM32每个定时器最多可输出几路PWM？STM32的定时器总共可输出几路PWM？

解：每个通用/高级定时器有4个通道（CH1~CH4），可输出4路独立PWM。

高级定时器TIM1、TIM8各有4路=8路，通用TIM2~TIM5各4路=16路

若无特殊限制，**每个定时器4路。**

答案：每个定时器最多4路PWM输出

3. 输入捕获

3.1 功能

- 测量外部信号的频率、周期、脉宽
- 记录信号跳变时的计数器值

3.2 配置步骤

1. 配置GPIO（浮空输入/上拉输入）
2. 配置定时器（PSC、ARR）
3. 配置CCMRx（输入捕获映射、滤波、分频）
4. 配置CCER（极性与使能）
5. 使能捕获中断（DIER）
6. 使能定时器

例题：使用TIM2_CH1测量脉冲宽度，简述配置流程。

答案：

- ① PA0（TIM2_CH1）配置为浮空输入
- ② TIM2的PSC分频使计数器频率合适
- ③ CCMR1配置CC1S=01（映射到TI1）
- ④ CCER配置CC1P=0（上升沿捕获）
- ⑤ 使能CC1中断
- ⑥ 使能TIM2
- ⑦ 在中断中读取CCR1值

4. 编码器接口模式

- 使用TI1和TI2两路输入
- 可测量位置、方向、速度
- 支持**×1、×2、×4** 倍频模式

模式对应关系

有效边沿	计数方向
TI1上升沿时TI2=高 → 向上	TI1上升沿时TI2=低 → 向下
TI2上升沿时TI1=高 → 向上	TI2上升沿时TI1=低 → 向下

例题：STM32定时器编码器模式支持几倍频？编码器模式的优势是什么？

答案：支持×1、×2、×4三种倍频模式。优势：

- ① 可确定旋转方向
- ② 可测量转速和位置
- ③ 具有抗干扰能力（毛刺抑制）
- ④ 硬件自动处理，不占CPU资源

5. 中断系统

5.1 STM32中断分类

类型	描述
外部中断EXTI	引脚边沿触发
定时器中断	更新/捕获/比较事件
串口中断	TXE/RXNE/TC

类型	描述
I2C/SPI中断	传输事件
ADC/DAC中断	转换完成

5.2 中断配置步骤

1. 开启外设时钟
2. 配置GPIO（若需外部中断）
3. 配置NVIC（优先级分组、抢占/子优先级）
4. 配置外设中断条件（如上升沿/下降沿）
5. 使能中断
6. 编写ISR（中断服务函数）

例题：配置PA0为外部中断（上升沿触发），NVIC抢占优先级2，子优先级1，写出配置代码框架。

解：

```
// 1. 开启时钟
RCC->APB2ENR |= (1 << 2); // GPIOA
RCC->APB2ENR |= (1 << 0); // AFIO

// 2. 配置GPIO
GPIOA->CRL &= ~0x0F; // PA0浮空输入

// 3. 配置EXTI
AFIO->EXTICR[0] &= ~0x0F; // EXTI0连接到PA0
EXTI->IMR |= (1 << 0); // 不屏蔽EXTI0
EXTI->RTSR |= (1 << 0); // 上升沿触发

// 4. 配置NVIC（分组2：2位抢占 + 2位子优先）
NVIC_SetPriorityGrouping(2); // AIRCR[10:8]=010 → 分组2
NVIC_SetPriority(EXTI0_IRQn, (2<<2) | 1); // 抢占2，子1，优先级值=0x90
NVIC_EnableIRQ(EXTI0_IRQn);
```

6. 串口 (USART)

6.1 基本参数

- **波特率**: 通信速率 (bps)
- **数据位**: 8/9位
- **停止位**: 0.5/1/1.5/2位
- **校验位**: 无/奇/偶校验

6.2 波特率计算公式

USART_BRR = 外设时钟 / (16 × 目标波特率)

例题： PCLK2=72MHz，目标波特率115200bps，求USART_BRR值。

解：

$USART_BRR = 72,000,000 / (16 \times 115200) = 72M / 1843200 \approx 39.0625$

$39 = 0x27$ ，小数部分 $0.0625 \times 16 = 1 \rightarrow DIV_Fraction = 1$

$USART_BRR = (39 << 4) | 1 = 0x0271$

答案： 0x0271

6.3 USART中断

中断标志	含义
TXE	发送数据寄存器空
TC	发送完成
RXNE	接收数据寄存器非空
IDLE	总线空闲
PE	校验错误

例题： USART的中断标志位TXE和TC有什么区别？

答案： TXE表示数据已从DR移入移位寄存器，可以写入新数据；TC表示数据已从移位寄存

器完全发送完毕（需等待最后一个停止位结束）。TC通常用于RS485方向控制等需要精确等待发送完成的场景。

7. ADC（模数转换器）

7.1 STM32 ADC特点

- 12位逐次逼近型ADC
- 最多3个ADC模块（ADC1/2/3）
- 18个通道（16个外部+2个内部）
- 多种触发方式：软件、定时器、外部引脚、其它ADC

7.2 转换时间

$T_{\text{conversion}} = \text{采样周期} + 12.5 \text{个ADC时钟周期}$

ADC时钟 $\leq 14\text{MHz}$

例题：ADC时钟为14MHz，采样周期为1.5个时钟周期，求单次转换时间。

解：

$T_{\text{conversion}} = (1.5 + 12.5) / 14\text{MHz} = 14 / 14\text{MHz} = 1\mu\text{s}$

转换速率 = $1 / 1\mu\text{s} = 1\text{MSPS}$

答案：1μs, 1MSPS（每秒百万次采样）

8. DMA（直接存储器访问）

- 无需CPU参与直接在存储器与外设之间传输数据
- STM32F103有2个DMA控制器（DMA1有7个通道，DMA2有5个通道）
- 传输方向：外设→存储器、存储器→外设、存储器→存储器

DMA配置要素

- 源地址、目的地址
- 传输数据量
- 传输方向
- 地址增量模式（是否递增）
- 传输完成中断

例题：DMA的优势是什么？使用DMA传输ADC转换结果的过程是怎样的？

答案：DMA的优势是无需CPU干预即可完成数据传输，节省CPU资源。ADC+DMA的过程：

- ① 配置ADC为连续转换模式
- ② 配置DMA，源地址为ADC->DR，目的地址为内存数组
- ③ 设置传输数据量和循环模式
- ④ 启动ADC和DMA
- ⑤ 每次转换完成，DMA自动将结果搬运到内存数组

9. SPI（串行外设接口）

- **全双工同步通信**
- **4线制：**SCK（时钟）、MOSI（主出从入）、MISO（主入从出）、CS（片选）
- **4种模式**（由CPOL和CPHA决定）
 - CPOL：时钟极性（空闲电平）
 - CPHA：时钟相位（数据采样边沿）

SPI模式表

模式	CPOL	CPHA	采样边沿
0	0	0	第一个跳变沿采样
1	0	1	第二个跳变沿采样
2	1	0	第一个跳变沿采样

模式	CPOL	CPHA	采样边沿
3	1	1	第二个跳变沿采样

例题：SPI的四种工作模式如何区分？简述SPI和I2C的区别。

答案：

SPI四种模式由CPOL（时钟极性）和CPHA（时钟相位）决定：

- CPOL=0：空闲时SCK为低电平
- CPOL=1：空闲时SCK为高电平
- CPHA=0：第一个边沿采样
- CPHA=1：第二个边沿采样

SPI与I2C区别：

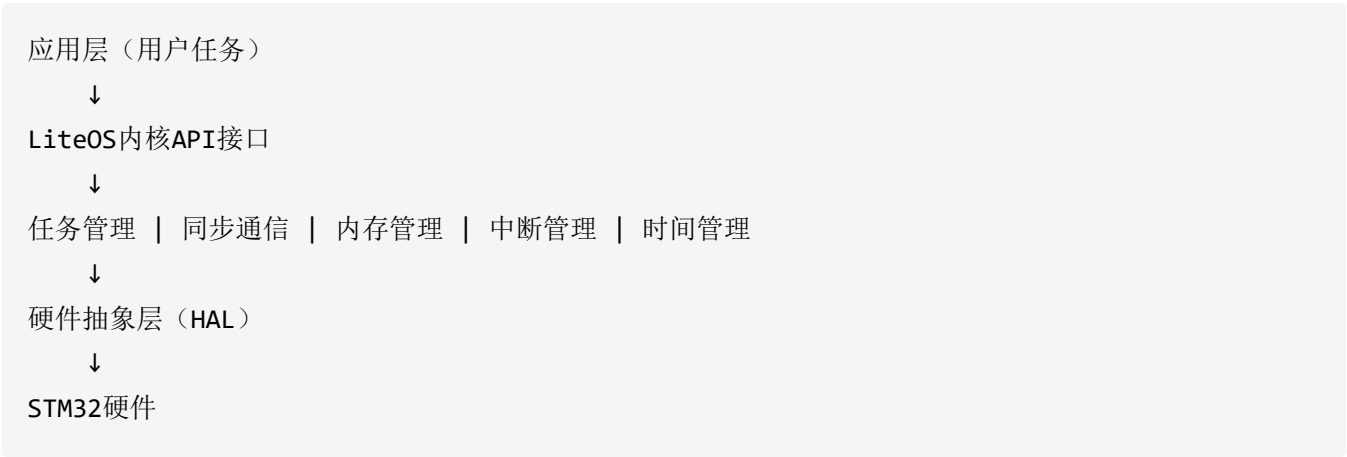
- SPI：全双工，4线，速率高，无应答，多设备需多用CS
- I2C：半双工，2线（SCL+SDA），速率较低，有应答机制，地址寻址

第七部分 LiteOS 基础

1. LiteOS 简介

- **轻量级开源IoT操作系统**
- 支持ARM Cortex-M/R等架构
- **极低内核**：最小RAM约1KB，ROM约4KB
- 支持**任务调度、同步、通信、内存管理、中断管理**

2. 内核架构



3. 系统时钟和Tick

- `LOSCFG_BASE_CORE_TICK_PER_SECOND`：每秒Tick数
- Tick是操作系统的基本时间单位
- 系统通过SysTick定时器产生Tick中断

4. LiteOS 内核初始化流程

```
// 1. 硬件初始化（时钟、外设等）
// 2. 调用LOS_KernelInit()初始化内核
// 3. 创建用户任务
// 4. 调用LOS_Start()启动调度
```

例题： LiteOS内核初始化的关键步骤是什么？

答案：

- ① 调用 `LOS_KernelInit()` 初始化内核（内存池、任务队列等）
- ② 创建用户任务（如Test1_Task、Test2_Task）
- ③ 调用 `LOS_Start()` 启动任务调度
- ④ 系统开始运行最高优先级的就绪任务

第八部分 LiteOS 任务概述

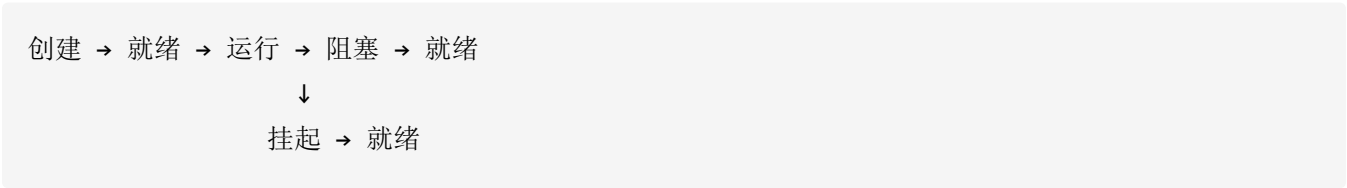
1. 任务特性

- 任务是操作系统调度的最小单位
- 每个任务拥有独立的**任务栈**
- 任务具有**优先级**
- 任务通过**任务控制块（TCB）** 管理

2. 任务状态

状态	含义
就绪（Ready）	任务已就绪，等待CPU
运行（Running）	任务正在执行
阻塞（Blocked）	任务等待事件/资源
挂起（Suspended）	任务被主动挂起（不参与调度）
未初始化（Uninit）	任务未创建

状态转换图



3. 任务优先级

- 数值越小，优先级越高
- 优先级范围：0 ~ (LOSCFG_BASE_CORE_TSK_PRIORITY_LIMIT - 1)
- 相同优先级任务按**时间片轮转**调度（若使能了LOSCFG_BASE_CORE_TIMESLICE）

例题： LiteOS任务有哪几种状态？简述状态转换条件。

答案：

五状态：就绪、运行、阻塞、挂起、未初始化。

- 就绪→运行：调度器选择最高优先级就绪任务
- 运行→就绪：任务被更高优先级任务抢占/时间片用完
- 运行→阻塞：任务等待资源或延时
- 阻塞→就绪：等待条件满足
- 运行→挂起：主动调用挂起函数
- 挂起→就绪：被其他任务恢复

第九部分 LiteOS 任务创建

1. 任务创建API

```
UINT32 LOS_TaskCreate(  
    UINT32 *puwTaskID,           // 任务ID输出指针  
    TSK_INIT_PARAM_S *pstInitParam // 初始化参数  
);
```

2. 任务初始化参数结构体

```
typedef struct {  
    TSK_ENTRY_FUNC pfnTaskEntry; // 任务入口函数  
    UINT16 usTaskPrio;           // 任务优先级  
    UINT32 uwStackSize;         // 任务栈大小  
    UINT32 *puvStackAddr;       // 栈起始地址(NULL为自动分配)  
    UINT32 uwArg;               // 入口函数参数  
    CHAR *pcName;               // 任务名  
    UINT32 uwResved;            // 保留  
} TSK_INIT_PARAM_S;
```

3. 任务创建示例

```
UINT32 Test1_Task_ID;
TSK_INIT_PARAM_S g_stTask1;

UINT32 Creat_Test1_Task(void)
{
    UINT32 uwRet = LOS_OK;
    g_stTask1.pfnTaskEntry = (TSK_ENTRY_FUNC)Test1; // 入口函数
    g_stTask1.usTaskPrio = 5; // 优先级5
    g_stTask1.uwStackSize = 0x1000; // 4KB栈
    g_stTask1.pcName = "Test1_Task";
    g_stTask1.uwArg = 0;

    uwRet = LOS_TaskCreate(&Test1_Task_ID, &g_stTask1);
    if (uwRet != LOS_OK) {
        printf("Test1_Task创建失败!\n");
    }
    return uwRet;
}

VOID Test1(void)
{
    while (1) {
        printf("Test1 Task Running\n");
        LOS_TaskDelay(1000); // 延时1000个Tick
    }
}
```

例题：创建一个优先级为3、栈大小为512字节、任务名为"MyTask"的任务，写出完整代码。

解：

```

UINT32 MyTask_ID;
TSK_INIT_PARAM_S g_stMyTask;

VOID MyTask_Entry(void)
{
    while (1) {
        printf("MyTask Running\n");
        LOS_TaskDelay(500);
    }
}

UINT32 Creat_MyTask(void)
{
    UINT32 uwRet;
    g_stMyTask.pfnTaskEntry = (TSK_ENTRY_FUNC)MyTask_Entry;
    g_stMyTask.usTaskPrio = 3;
    g_stMyTask.uwStackSize = 0x200; // 512字节
    g_stMyTask.pcName = "MyTask";
    g_stMyTask.uwArg = 0;

    uwRet = LOS_TaskCreate(&MyTask_ID, &g_stMyTask);
    return uwRet;
}

```

第十部分 LiteOS 任务同步与进程间通信

1. 信号量 (Semaphore)

1.1 信号量分类

类型	取值	用途
计数型信号量	0~N	资源计数，控制多个资源访问

类型	取值	用途
二值信号量	0/1	互斥访问或任务同步

1.2 信号量API

```
// 创建信号量
UINT32 LOS_SemCreate(UINT16 usCount, UINT32 *puwSemHandle);

// P操作（获取/等待）
UINT32 LOS_SemPend(UINT32 uwSemHandle, UINT32 uwTimeout);

// V操作（释放）
UINT32 LOS_SemPost(UINT32 uwSemHandle);

// 删除信号量
UINT32 LOS_SemDelete(UINT32 uwSemHandle);
```

1.3 典型同步模式



例题：使用信号量实现任务A和任务B的同步，任务A等待任务B处理完数据后再继续。

解：

```
UINT32 g_uwSemHandle;

// 初始化：创建二值信号量，初始为0
LOS_SemCreate(0, &g_uwSemHandle);

// 任务A（等待者）
VOID TaskA(void)
{
    while (1) {
        LOS_SemPend(g_uwSemHandle, LOS_WAIT_FOREVER); // 等待B完成
        printf("A: 收到信号，继续工作\n");
    }
}

// 任务B（释放者）
VOID TaskB(void)
{
    while (1) {
        LOS_TaskDelay(1000);           // 模拟数据处理
        LOS_SemPost(g_uwSemHandle);    // 释放信号量通知A
        printf("B: 处理完成，通知A\n");
    }
}
```

2. 互斥锁（Mutex）

2.1 特点

- 二值信号量的特殊形式
- 支持**优先级继承**，防止优先级反转
- 用于保护共享资源（临界区）

2.2 API

```
UINT32 LOS_MuxCreate(UINT32 *puwMuxHandle);
UINT32 LOS_MuxPend(UINT32 uwMuxHandle, UINT32 uwTimeout);
UINT32 LOS_MuxPost(UINT32 uwMuxHandle);
```

例题：使用互斥锁保护全局变量的访问。

解：

```
UINT32 g_uwMuxHandle;
int g_shared_data = 0;

VOID Task_Write(void)
{
    while (1) {
        LOS_MuxPend(g_uwMuxHandle, LOS_WAIT_FOREVER);
        g_shared_data++; // 安全访问共享资源
        LOS_MuxPost(g_uwMuxHandle);
        LOS_TaskDelay(100);
    }
}

VOID Task_Read(void)
{
    int val;
    while (1) {
        LOS_MuxPend(g_uwMuxHandle, LOS_WAIT_FOREVER);
        val = g_shared_data; // 安全读取共享资源
        LOS_MuxPost(g_uwMuxHandle);
        printf("数据: %d\n", val);
        LOS_TaskDelay(500);
    }
}
```

2.3 优先级反转与优先级继承

- **优先级反转：**低优先级任务持有锁，高优先级任务等待，中优先级任务正常运行导致高优先

级任务无法执行

- **优先级继承**：持有锁的低优先级任务临时提升到等待该锁的最高优先级任务的级别，解决反转问题

例题：什么是优先级反转？互斥锁如何解决优先级反转问题？

答案：优先级反转是指低优先级任务持有高优先级任务所需的资源，导致高优先级任务被阻塞，而中优先级任务不受影响持续运行，使得高优先级任务迟迟无法执行。

互斥锁的**优先级继承**机制：当高优先级任务等待一个被低优先级任务持有的锁时，该低优先级任务被临时提升到高优先级任务的级别，从而不会被中优先级任务抢占，尽快释放锁，然后恢复原优先级。

3. 消息队列（Message Queue）

3.1 特点

- 任务间或中断与任务间通信
- 先进先出（FIFO）
- 支持**发送/接收**定长消息

3.2 创建消息队列

```
UINT32 LOS_QueueCreate(  
    CHAR *pcQueueName,           // 队列名（暂未使用）  
    UINT16 usLen,                 // 队列长度（消息数）  
    UINT32 *puwQueueID,          // 队列ID输出  
    UINT32 uwFlags,               // 标志（暂未使用）  
    UINT16 usMaxMsgSize          // 最大消息大小（字节）  
);
```

3.3 发送和接收

```
// 发送消息（带超时）
UINT32 LOS_QueueWrite(UINT32 uwQueueID, VOID *pBuffer,
                      UINT32 uwBufferSize, UINT32 uwTimeout);

// 接收消息
UINT32 LOS_QueueRead(UINT32 uwQueueID, VOID *pBuffer,
                     UINT32 uwBufferSize, UINT32 uwTimeout);
```

3.4 创建示例

```
UINT32 Queue_Handle;

// 创建一个长度为128、最大消息大小为16字节的队列
uwRet = LOS_QueueCreate("Test_Queue", 128, &Queue_Handle, 0, 16);
if (uwRet != LOS_OK) {
    printf("队列创建失败\n");
}
```

例题：使用消息队列实现任务A发送数据给任务B。

解：

```
UINT32 g_QueueID;
#define MSG_SIZE 16

VOID TaskA_Sender(void)
{
    UINT32 uwRet;
    char buf[MSG_SIZE];
    int cnt = 0;
    while (1) {
        sprintf(buf, "Msg_%d", cnt++);
        uwRet = LOS_QueueWrite(g_QueueID, buf, MSG_SIZE, LOS_WAIT_FOREVER);
        if (uwRet == LOS_OK) printf("A: 发送 %s\n", buf);
        LOS_TaskDelay(500);
    }
}

VOID TaskB_Receiver(void)
{
    UINT32 uwRet;
    char buf[MSG_SIZE];
    while (1) {
        uwRet = LOS_QueueRead(g_QueueID, buf, MSG_SIZE, LOS_WAIT_FOREVER);
        if (uwRet == LOS_OK) printf("B: 收到 %s\n", buf);
    }
}

// 主函数创建队列
VOID main(void)
{
    LOS_QueueCreate("Q", 10, &g_QueueID, 0, MSG_SIZE);
    // 创建任务A和B...
}
```

4. 事件 (Event)

4.1 特点

- **多对多同步**：一个任务可等待多个事件
- 每个事件用一个bit表示（32位共32个事件）
- 支持**与模式**（所有条件满足）和**或模式**（任一条件满足）

4.2 API

```
// 创建事件
UINT32 LOS_EventCreate(PEVENT_CB_S pstEventCB);

// 等待事件
UINT32 LOS_EventPend(PEVENT_CB_S pstEventCB, UINT32 uwEventMask,
                    UINT32 uwMode, UINT32 uwTimeout);

// 发送事件
UINT32 LOS_EventWrite(PEVENT_CB_S pstEventCB, UINT32 uwEvents);

// 清除事件
UINT32 LOS_EventClear(PEVENT_CB_S pstEventCB, UINT32 uwEvents);
```

4.3 等待模式

- **LOS_WAITMODE_AND**：所有指定事件位都发生才返回
- **LOS_WAITMODE_OR**：任意一个指定事件位发生就返回
- **LOS_WAITMODE_CLR**：事件发生后自动清除事件位

例题：使用事件实现任务C等待任务A和任务B都完成后再执行。

解：

```
EVENT_CB_S g_stEventCB;
#define EVENT_A (1 << 0) // bit0: 任务A完成
#define EVENT_B (1 << 1) // bit1: 任务B完成

VOID TaskA(void)
{
    LOS_TaskDelay(500);
    LOS_EventWrite(&g_stEventCB, EVENT_A);
    printf("A完成\n");
}

VOID TaskB(void)
{
    LOS_TaskDelay(1000);
    LOS_EventWrite(&g_stEventCB, EVENT_B);
    printf("B完成\n");
}

VOID TaskC(void)
{
    while (1) {
        // 等待A和B都完成（AND模式）
        UINT32 uwRet = LOS_EventPend(&g_stEventCB, EVENT_A | EVENT_B,
                                     LOS_WAITMODE_AND, LOS_WAIT_FOREVER);

        if (uwRet == LOS_OK) {
            printf("A和B都完成了，C开始执行\n");
        }
    }
}
```

总复习总结：考点与例题全覆盖

考点1：数制转换与编码

重要度：☆☆☆

知识点：二进制/十六进制/十进制转换、原码反码补码、有符号数表示

例题1-1：将十进制数-13用8位二进制补码表示。

解： $13D=00001101B \rightarrow$ 取反 $11110010B \rightarrow +1=11110011B$

答案：11110011B

例题1-2：十六进制4D3E转换为二进制。

解： $4 \rightarrow 0100, D \rightarrow 1101, 3 \rightarrow 0011, E \rightarrow 1110$

答案：0100 1101 0011 1110B

考点2：逻辑运算与位操作

重要度：☆☆☆

知识点：与/或/非/异或的真值表、按位操作应用（置位/清零/取反）

例题2-1： $X=1100B, Y=1010B$ ，求 $X \& Y$ 、 $X | Y$ 、 $X \wedge Y$ 、 $\sim X$ 。

答案： $1100 \& 1010 = 1000B$ ； $1100 | 1010 = 1110B$ ； $1100 \wedge 1010 = 0110B$ ； $\sim 1100 = 0011B$

例题2-2：如何将寄存器R的第3位置1？第4位清零？

答案：置位： $R |= (1 << 3)$ ；清零： $R \&= \sim(1 << 4)$

考点3：微型计算机硬件结构

重要度：☆☆

知识点：CPU组成（ALU/CU/寄存器）、三总线结构、存储器组织

例题3-1： CPU由哪些部分组成？各部件功能是什么？

答案： ①ALU：算术逻辑运算 ②CU：指令译码与控制 ③寄存器组：暂存数据。
详细包括：累加器A、暂存器TMP、ALU、标志寄存器F、程序计数器PC等。

例题3-2： n根地址线的寻址范围是？某CPU有16根地址线，8根数据线，求寻址范围和数据宽度。

答案： 寻址范围= 2^n 字节。16根地址线 $\rightarrow 2^{16}=64\text{KB}$ 寻址范围；8根数据线 $\rightarrow$ 8位数据宽度（1字节）

考点4：Cortex-M3寄存器与异常

重要度：☆☆☆☆

知识点：寄存器组（R0~R15, xPSR, PRIMASK）、异常类型与优先级、自动压栈、NVIC

例题4-1： Cortex-M3进入异常时硬件自动压栈哪些寄存器？占多少字节？

答案： 自动压栈8个寄存器：xPSR、PC、LR、R12、R3、R2、R1、R0。每个32位即4字节，共32字节。

例题4-2： NVIC->IPR[1]对应中断4~7，值为0x70806050，求优先级顺序。

答案： IPR[1]字节排列为[7:0] $\rightarrow$ IRQ4、[15:8] $\rightarrow$ IRQ5、[23:16] $\rightarrow$ IRQ6、[31:24] $\rightarrow$ IRQ7
IRQ4=0x50(80)，IRQ5=0x60(96)，IRQ6=0x80(128)，IRQ7=0x70(112)
数值小优先级高：**IRQ4 > IRQ5 > IRQ7 > IRQ6**

考点5：STM32时钟系统

重要度：☆☆☆☆

知识点：HSI/HSE/PLL、SYSCLK选择、AHB/APB分频、时钟树计算、CSS

例题5-1： SYSCLK=72MHz，AHB预分频HPRE=1011B，求HCLK。

解： 1011B→SYSCLK/16分频。HCLK=72MHz/16=4.5MHz

答案： 4.5MHz

例题5-2： CSS的功能是什么？

答案： CSS监测HSE（外部高速晶振），当HSE失效时自动切换到HSI（8MHz），并产生NMI中断。

考点6：GPIO配置与操作

重要度：☆☆☆☆

知识点：8种工作模式、CRL/CRH配置、ODR/BSRR/IDR操作、推挽与开漏区别

例题6-1： 配置PA5为推挽输出（50MHz），写出代码。

解：

```
RCC->APB2ENR |= (1<<2);    // GPIOA时钟使能
GPIOA->CRL &= ~(0xF<<20);    // 清零PA5[4位]
GPIOA->CRL |= (0x3<<20);      // MODE=11(50MHz) CNF=00(推挽)
// 输出高电平
GPIOA->BSRR = (1<<5);
```

例题6-2： 浮空输入、上拉输入、下拉输入的区别是什么？

答案：

- 浮空输入：无内部上拉/下拉，引脚高阻态，电平由外部决定

- 上拉输入：内部上拉到VDD，引脚默认高电平
- 下拉输入：内部下拉到VSS，引脚默认低电平

考点7：定时器与PWM

重要度：☆☆☆☆

知识点：PSC/ARR/CCR计算、溢出时间公式、PWM频率占空比计算、编码器模式

例题7-1：TIM3时钟=36MHz，PSC=7199，ARR=4999，求溢出时间。

解：计数频率=36MHz/(7199+1)=36M/7200=5kHz

溢出时间=(4999+1)/5kHz=1s

答案：1秒

例题7-2：用TIM2产生1kHz、60%占空比的PWM，PCLK1=36MHz，求PSC、ARR、CCR2。

解：取PSC=3599→计数频率=36M/3600=10kHz

ARR=10kHz/1kHz-1=9；CCR2=ARR×60%=5.4≈5

答案：PSC=3599，ARR=9，CCR2=5（实际占空比≈55.6%）

考点8：串口USART

重要度：☆☆

知识点：波特率计算、USART_BRR公式、中断标志位

例题8：PCLK=72MHz，波特率=9600，求USART_BRR值。

解：USART_BRR = 72,000,000/(16×9600) = 72,000,000/153,600 = 468.75

整数部分=468=0x1D4，小数部分=0.75×16=12=0xC

USART_BRR = (0x1D4<<4) | 0xC = 0x1D4C

答案：0x1D4C

考点9：ADC与DMA

重要度：☆☆☆

知识点：ADC分辨率、转换时间、DMA优势与配置

例题9：STM32的ADC转换时间如何计算？若ADC_CLK=14MHz，采样周期=7.5周期，求转换时间。

解： $T_{\text{conv}} = \text{采样周期} + 12.5 = 7.5 + 12.5 = 20$ 个ADC时钟周期

$T_{\text{conv}} = 20 / 14\text{MHz} \approx 1.43\mu\text{s}$

答案：约1.43μs

考点10：LiteOS任务管理

重要度：☆☆☆☆

知识点：任务五状态、优先级、任务创建（API）、任务控制块、状态转换

例题10-1：画图说明LiteOS任务的5种状态及转换关系。

答案：

创建(Uninit) → 就绪(Ready) → 运行(Running) → 阻塞(Blocked) → 就绪

↓

挂起(Suspended) → 就绪

转换条件：就绪→运行（调度器选择）；运行→就绪（被抢占/时间片完）；

运行→阻塞（等待资源/延时）；阻塞→就绪（条件满足）；运行→挂起（主动挂起）；挂起→就绪（被恢复）

例题10-2：创建一个优先级为2、栈大小1024字节、任务名为"Task_LED"的任务。

解：

```
UINT32 Task_LED_ID;
TSK_INIT_PARAM_S g_stTaskLED;

VOID Task_LED_Entry(void)
{
    while(1) {
        printf("LED Task Running\n");
        LOS_TaskDelay(200);
    }
}

UINT32 Create_LED_Task(void)
{
    g_stTaskLED.pfnTaskEntry = (TSK_ENTRY_FUNC)Task_LED_Entry;
    g_stTaskLED.usTaskPrio = 2;
    g_stTaskLED.uwStackSize = 0x400; // 1024字节
    g_stTaskLED.pcName = "Task_LED";
    g_stTaskLED.uwArg = 0;
    return LOS_TaskCreate(&Task_LED_ID, &g_stTaskLED);
}
```

考点11： LiteOS任务同步与通信

重要度：☆☆☆☆

知识点：信号量（P/V操作）、互斥锁（优先级继承）、消息队列（FIFO）、事件（与/或模式）

例题11-1：信号量LOS_SemPend和LOS_SemPost的作用是什么？

答案：

- LOS_SemPend：P操作，获取信号量。若信号量>0则减1后返回；若=0则任务阻塞等待（可设置超时）
- LOS_SemPost：V操作，释放信号量。将信号量+1，唤醒等待该信号量的最高优先级任务

例题11-2：使用事件实现任务C等待任务A或B任意一个完成后执行。

解：

```
EVENT_CB_S g_stEventCB;
#define EV_A (1<<0)
#define EV_B (1<<1)

VOID TaskC(void)
{
    LOS_EventPend(&g_stEventCB, EV_A|EV_B,
                  LOS_WAITMODE_OR, LOS_WAIT_FOREVER);
    printf("A或B已完成\n");
}
```

例题11-3：什么是优先级反转？互斥锁如何解决？

答案：优先级反转指低优先级任务持有高优先级任务所需的锁，而中优先级任务抢占CPU，导致高优先级任务无法执行。互斥锁的**优先级继承**机制：持有锁的低优先级任务临时提升到高优先级任务的级别执行，防止被中优先级任务抢占，释放锁后恢复原优先级。

考点12：大小端与存储器

重要度：☆☆

知识点：大小端区别、STM32默认小端、存储器映射

例题12：32位数据0x12345678以小端存储于起始地址0x20000000，写出各字节内容。

答案：0x20000000: 78H, 0x20000001: 56H, 0x20000002: 34H, 0x20000003: 12H

考点13：中断系统

重要度：☆☆☆

知识点：NVIC配置、抢占优先级/子优先级分组、外部中断配置

例题13：若AIRCRR[10:8]=011（分组3），某中断占用优先级寄存器的值为0x90，求其抢占优先级和子优先级。

解：分组3：抢占优先级3位，子优先级1位
0x90=1001_0000B → 抢占:100B=4，子:1B=1

答案：抢占优先级=4，子优先级=1

考试出题规律总结

试卷结构推测（总分70分理论）

题型	分值	题量	特点
选择题	2030分	10~15题	基础概念、简单计算
填空题	1015分	5~10空	关键参数、寄存器名、状态
简答题	1520分	3~5题	原理描述、区别比较、状态转换
编程/ 计算题	1520分	2~4题	时钟计算、定时器配置、GPIO配置、LiteOS任务创建

高频考点命中率

考点	预计出现概率	常见题型
GPIO配置（CRL/BSRR）	★★★★★	选择+编程
时钟树计算（SYSCLK→HCLK→PCLK）	★★★★★	计算/选择
定时器PSC/ARR/溢出时间	★★★★★	计算
CM3寄存器/自动压栈	★★★★	选择/填空
NVIC中断优先级	★★★★	选择/计算

考点	预计出现概率	常见题型
数制/补码/逻辑运算	★★★★	选择/填空
LiteOS任务状态转换	★★★★	简答
信号量/互斥锁/消息队列	★★★★	简答/编程
大小端模式	★★★★	选择
PWM占空比/频率	★★★★	计算
推挽/开漏区别	★★★★	选择/简答
原码反码补码	★★★★	选择/填空

答题技巧

1. **计算题**先写公式再代入数值，步骤分很重要
2. **时钟计算**注意APB1最高36MHz、APB2最高72MHz的限制
3. **补码计算**注意正数补码=原码，负数补码=反码+1
4. **GPIO配置**注意CRL(07)和CRH(815)的位数对应关系
5. **定时器**公式： $f_cnt = f_timer / (PSC + 1)$ ， $T_overflow = (ARR + 1) / f_cnt$
6. **中断优先级**数值越小优先级越高
7. **LiteOS任务**创建必须包含入口函数、优先级、栈大小三个核心参数
8. **信号量**创建时usCount=0用于同步，=1用于互斥

✦ **复习建议：** 优先掌握考点5★★★★（时钟系统）、考点6★★★★（GPIO）、考点7★★★★（定时器PWM）、考点10★★★★（LiteOS任务）、考点11★★★★（LiteOS同步通信）。这些是考试的重中之重，几乎每年必考。